

Prompt Input Formats: Plain Text vs. JSON/YAML/Markdown/XML

For feeding instructions to an LLM, structured formats generally improve reliability but vary widely in token cost. For example, one study found that **JSON schemas** dramatically boost parse accuracy and consistency, acting like an “API contract” for the model ¹. However, JSON is verbose; **YAML** can save ~18% in tokens by using indentation instead of braces ². In contrast **XML** is highly expensive (requiring 14% more tokens than JSON and ~114% more than a compact “TOON” format) because every tag doubles in context ³. For large uniform datasets (e.g. tabular records), the new TOON notation is most token-efficient ⁴.

Markdown (plain text with headings/lists) has no native schema but greatly aids readability; using headings and bullet points “creates cleaner, more readable prompts” and helps the model parse complex instructions ⁵ ⁶. In practice, many projects use JSON or YAML for machine-readable configs and reserve Markdown/HTML for human-friendly prompt templates.

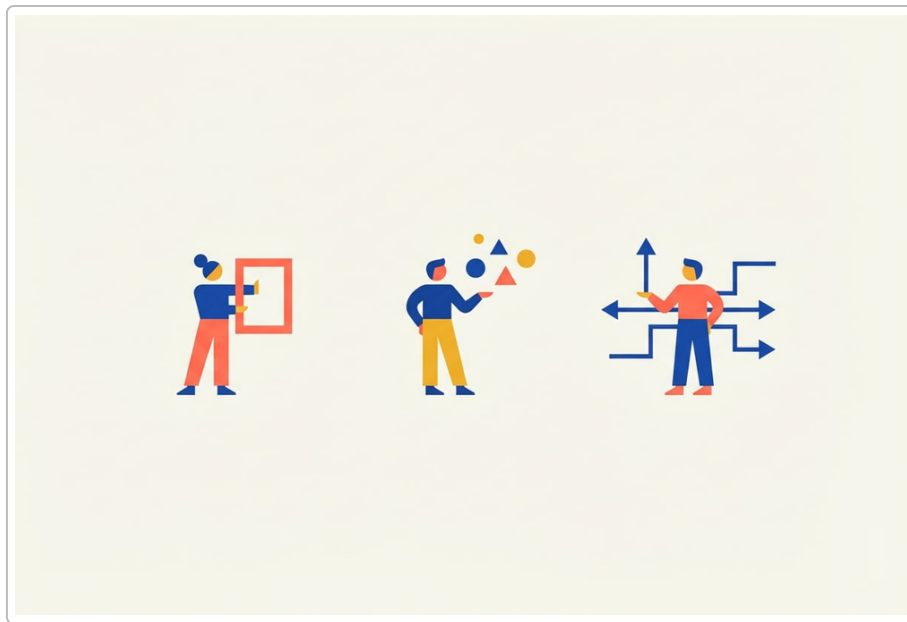
- **JSON (structured):** Explicit keys (task, context, format) make instructions unambiguous ¹. Enforcing field order can even improve reasoning (e.g. put “reasoning” before “answer” ⁷). Constrained-decoding tools can guarantee valid JSON output.
- **YAML (semi-structured):** More concise than JSON (~18% token savings) ², human-editable with comments. Beware indentation errors (“Norway”/“Yes” pitfalls) and parser security issues ⁸. Good for config schemas but slightly riskier for LLM output.
- **Markdown/Plain Text:** No rigid schema, but using **Markdown formatting** (headings, lists, bold) clearly delineates steps or sections ⁵ ⁶. Atlassian and others advise writing in natural conversational language with persona/task/context hints ⁹ ¹⁰, which often means a Markdown-like prompt.
- **XML:** Highly verbose (each `<tag>` and `</tag>` adds cost) ³. Its semantic tags can make sections clear, but the token overhead is very high. XML is sometimes used for human-edited prompt templates (due to its explicit tag structure) ³, but it’s usually avoided for raw LLM input.

In summary, use **structured JSON/YAML** when you need precise, machine-readable instructions (especially for output parsing) ¹. Use **Markdown/clearly-worded text** to provide context and ease human editing, and rely on headings or bullet lists to organize complex tasks ⁵. Avoid XML for performance unless you specifically need its tag semantics despite the cost ³.

AI Coding Tools: Editor Assistants vs Autonomous Agents

Different AI coding tools reflect two philosophies: **in-editor assistants** versus **autonomous CLI agents**. Cursor and Windsurf are AI-powered **VS Code-based IDEs** (forked from VS Code) that “bolt AI” into the editor interface ¹¹. They offer features like inline code completion, chat or “tab” modes, and special multi-step agent flows (e.g. Windsurf’s *Cascade* mode). In contrast, Claude Code (by Anthropic) is a **terminal-based autonomous agent**: it runs entirely from the command line, reads/writes files, and can execute multi-step plans without a GUI ¹¹. GitHub Copilot occupies both worlds: it primarily appears as a **plugin**

for IDEs (VS Code, JetBrains, Neovim, etc.) but also provides a **CLI mode**. As one analysis notes, “Copilot’s breadth... as a standalone CLI” plus editor integrations is a “genuine enterprise advantage” ¹² .



Cursor and Windsurf integrate AI directly into the VS Code interface (Windsurf’s “Cascade” agent can even infer context across files), whereas Claude Code operates as a standalone CLI agent ¹¹ . GitHub Copilot, by contrast, runs as a plugin for common IDEs and provides a CLI; this makes it an inline coding assistant rather than a purely autonomous agent ¹² ¹³ . In short, tools like Cursor, Windsurf, and Copilot IDE editions act as **in-editor AI assistants**, while Claude Code (and Copilot CLI) act as **terminal-based agents** executing broader workflows.

- **Editor-embedded (AI Assistants):** Examples include **Cursor** and **Windsurf** (both deep VS Code forks) and **GitHub Copilot** in VS Code/JetBrains. They provide code completions, context-aware suggestions, and chat within the editor ¹¹ . These are best for day-to-day coding, refactoring, and navigating code (multi-file editing) because they have direct access to your workspace context.
- **CLI/Agentic Tools:** Examples include **Claude Code** and the **Copilot CLI** ¹² ¹³ . These run in the terminal and can autonomously perform tasks (create files, run commands, etc.). They are suited for larger planning tasks or CI/CD automation where an agent “works in the background” on a project ¹¹ .

The interfaces reflect different models: editor tools typically use completion/chat modes, while CLI agents rely on natural-language commands and “skills.” The table below (from a comparative review) summarizes the distinction in how these tools are delivered:

- **Interface:** Copilot uses IDE plugins (VS Code, JetBrains, etc.); **Claude Code** is **Terminal/CLI**; **Cursor** is a standalone VS Code fork; **Windsurf** runs on VS Code (+ JetBrains) ¹³ .
- **Typical Use:** Editor assistants excel at real-time coding help, multi-file refactors, and AI-native editing. CLI agents shine at autonomous, long-running tasks where they can plan and execute (e.g., massive refactor, codebase overhaul).

Effective Prompt Structuring

To get the best results from an LLM, structure your instructions carefully:

- **Use the Task-Context-Format (TCF) pattern:** Clearly state *what* to do, *why/for whom*, and *how to format the output* ¹⁴ ⁹. For example, specify “Task: build an ecommerce app”, provide context like “Context: The app should sell custom T-shirts, has target audience of hobbyists”, and then “Format: respond with a JSON schema of project details.” This “eliminates ambiguity” and tells the AI exactly what success looks like ¹⁴. Atlassian similarly advises including persona, task, context, and desired format in the prompt ⁹.
- **Give detailed but concise instructions:** State instructions in clear, natural language ¹⁰. More detail (without unnecessary fluff) yields more targeted results ¹⁰ ¹⁵. At the same time, avoid overly long, convoluted prompts that might confuse the model. Break instructions into bullet points or numbered steps if needed.
- **Provide examples (few-shot) if style matters:** When you have a specific style or content structure, giving 2–3 concrete examples helps a lot ¹⁶. The model can “see” exactly what you want. For instance, show example inputs/outputs or example lines of code/comment style and then ask it to mimic that pattern. Few-shot prompting is especially powerful for tone and format ¹⁶.
- **Chain-of-thought for complex reasoning:** For multi-step or analytical tasks, explicitly ask the model to “think step by step” and explain its reasoning ¹⁷. Kontent.ai notes that prompting the AI to “show its work” (listing reasoning before answering) produces deeper insight and more reliable conclusions ¹⁷. For example, ask “Explain your reasoning in steps before giving the final answer.” This often avoids surface-level answers.
- **Use Markdown or rich formatting:** Organize the prompt with Markdown headings, lists, or code blocks ⁵. Clear section breaks (e.g. “## Requirements”, “## Constraints”) help the model follow complex instructions. Syllable AI points out that when prompts are easier to read, “LLMs behave more consistently” and make fewer errors ⁵ ⁶.

In practice, a well-structured prompt might look like:

```
You are Senior E-commerce Architect building a new marketplace.

Task: Draft a system design document for an e-commerce web app with user
accounts, product listings, and checkout.

Context: The site is called "HobbyStuff", targeting craft enthusiasts. We
use a React front-end and Node.js back-end with a PostgreSQL database. The
codebase is in the `hobbystuff/` repo with Express and Sequelize.

Output Format: Provide an outline in markdown with sections for "Frontend
Components", "Backend APIs", "Database Schema". Use bullet lists for details.
```

This combines persona, task, context, and format explicitly.

Multi-Agent Team Framework

When treating multiple agents like a team of engineers, **orchestrate roles and context systematically**. In other words, decompose the project into specialized “agent engineers” and feed each agent the information it needs:

- **Define project metadata and context:** Begin by capturing your requirements in a structured form (JSON, YAML or Markdown). For example, write a project manifest that includes the **project name**, purpose, main features, technology stack (front-end framework, databases), and any constraints (e.g. budgets, deadlines). This metadata becomes the shared “brief” for the team. It can live in a file (like `project.json` or `REQUIREMENTS.md`) and provides context to all agents. Box’s multi-agent principles suggest that each agent has a clear role with a common goal ¹⁸. Your project spec is that common goal context.
- **Assign specialized agent roles:** Create an **Agent Reference** or skill files for each role, much like hiring a frontend dev, backend dev, QA, etc. For instance, you might have agents named `frontend-developer`, `backend-architect`, `database-admin`, and so on. One approach (as seen in open-source examples) is to put each agent’s definition in a file with YAML frontmatter:

```
name: frontend-developer
description: Builds UI in React/Angular, designs components
model: claude-sonnet
```

The `skills/` or `agents/` directory then contains each role’s system prompt (instructions on how to behave). Anthropic’s `SKILL.md` pattern works like this: each `SKILL.md` has a short metadata header and detailed instructions ¹⁹. By giving each agent a name/description and a model assignment, you ensure the right LLM is used for each specialist task ²⁰. (For example, wshobson’s agent repository defines 180+ roles like `python-pro`, `cloud-architect`, `security-auditor` with specific models ²¹ ²².)

- **Use context engineering:** Follow a context-management strategy so agents only see relevant information. Google’s Agent Development Kit (ADK) treats context as a “*compiled view*” of state ²³. In practice, this means keeping a shared **session log** of all dialogue and actions, plus any long-term **memory** of the project. Each agent invocation then gets a **working context** – a filtered view of the session (e.g. recent messages, tool outputs) and pertinent files. For instance, a backend agent might only receive the parts of the project description and code relevant to API design. Explicitly separating storage (session history, memory) from the prompt (working context) makes the system more reliable ²³ ²⁴. Tools like vector indexes over the codebase can help each agent retrieve only relevant code snippets.
- **Coordinate agents:** You can have one “chief” agent or human-in-the-loop to assign tasks. For example, a “conductor” agent reads the project goals and delegates subtasks (“Frontend: build the home page UI; Backend: design the authentication API”). Each specialist agent then uses its skill instructions to perform the task. Agents can also hand off to each other explicitly (the ADK framework even formalizes multi-agent handoffs ²³).

- **Example workflow:** Suppose you want an e-commerce app. You might start with a JSON spec:

```
{
  "project": "HobbyStuff Shop",
  "features": ["user accounts", "product catalog", "shopping cart",
    "checkout"],
  "tech": {"frontend": "React", "backend": "Node.js/Express",
    "db": "PostgreSQL"},
  "theme": "outdoor gear marketplace",
  "constraints": ["budget $10k", "deploy in 2 months"]
}
```

You feed this to a high-level agent (“product-manager” role) which breaks it into tasks. It says to `frontend-developer`: “Implement UI components for login, product listing, cart” and to `backend-architect`: “Design REST APIs for product and order endpoints” etc. Each agent has its SKILL prompt and the project metadata context to work from. They can even collaborate (the backend agent might ask the database agent for schema advice).

- **Leverage existing frameworks:** The agent reference project shows how roles might map to expertise. For instance, `frontend-developer` (Sonnet, React) and `backend-architect` (Opus, API design) are separate agents ²¹. Use the SKILL.md or agent plugin format: put each agent in `agents/{name}.md` with frontmatter and instructions ²⁰. This way your system treats agents as a team of engineers with clearly defined skills.

By combining structured project specs (JSON/markdown metadata) with a “team-of-agents” architecture, you give AI tools both the **big picture** and **specialist focus** they need. Each agent acts like a domain-specific engineer, and together they can construct the app or analyze an existing codebase. This multi-agent pattern (specialized roles, shared project context) follows best practices of MAS design ¹⁸ and scales much better than trying to rely on a single general LLM for everything.

Sources: Latest research and documentation on LLM prompting and AI agents were used. For example, structured JSON prompting is advocated as an “industrialized” standard ¹; Atlassian gives general prompt guidelines ⁹ ¹⁰; comparative reviews of code-AI tools detail their interfaces ¹¹ ¹²; the Google ADK blog explains context engineering for multi-agent systems ²³; and Box’s resources describe the multi-agent paradigm ¹⁸. All factual claims above are cited to these sources.

¹ Structured Prompting with JSON: The Engineering Path to Reliable LLMs | by vishal dutt | Medium

<https://medium.com/@vishal.dutt.data.architect/structured-prompting-with-json-the-engineering-path-to-reliable-llms-2c0cb1b767cf>

² ³ ⁴ ⁷ ⁸ LLM Structured Output: JSON, YAML, XML & TOON | Medium

<https://medium.com/@michael.hannecke/beyond-json-picking-the-right-format-for-llm-pipelines-b65f15f77f7d>

⁵ ⁶ Markdown Prompting Structure: Best Format for AI Agents

<https://syllable.ai/blog/ai-agent-markdown-prompt>

9 10 15 The ultimate guide to writing effective AI prompts - Work Life by Atlassian

<https://www.atlassian.com/blog/artificial-intelligence/ultimate-guide-writing-ai-prompts>

11 Windsurf vs Cursor vs Claude Code: Which AI Dev Tool Is Right for You? | MindStudio

<https://www.mindstudio.ai/blog/windsurf-vs-cursor-vs-claude-code>

12 13 GitHub Copilot vs Claude Code vs Cursor vs Windsurf (2026)

<https://kanerika.com/blogs/github-copilot-vs-claude-code-vs-cursor-vs-windsurf/>

14 16 17 The only 5 prompt structures you'll ever need | Kontent.ai

<https://kontent.ai/blog/the-only-5-prompt-structures-you-need/>

18 What are multi-agent systems? | Box

<https://www.box.com/resources/what-are-multi-agent-systems>

19 The SKILL.md Pattern: How to Write AI Agent Skills That Actually Work | by Bibek Poudel | Feb, 2026 | Medium

<https://bibek-poudel.medium.com/the-skill-md-pattern-how-to-write-ai-agent-skills-that-actually-work-72a3169dd7ee>

20 21 22 agents/docs/agents.md at main · wshobson/agents · GitHub

<https://github.com/wshobson/agents/blob/main/docs/agents.md>

23 24 Architecting efficient context-aware multi-agent framework for production - Google Developers Blog

<https://developers.googleblog.com/architecting-efficient-context-aware-multi-agent-framework-for-production/>